13 付録 — 逆引き: やりたいことからコードを探す

「一覧をしぼり込みたい」「数を 3 けたで区切りたい」から、書き方を 443 個引く。本文に出てくる所と、合わせて覚えるコードつき

13 付録: 逆引き — やりたいことからコードを探す

この逆引きについて

12 付録「出てくるコードとコマンドの一覧」は、「この `useEffect` は何だっけ？」のように、**コードから意味を引く**ための物です。この 13 付録は、その反対です。「**一覧を条件でしぼり込みたい**」「**数を 3 けたごとに区切って表示したい**」のような、**やりたいことから、書き方を引く**ための物です。

- **全部で 443 個**の「やりたいこと」を、**17 の分類**に分けています。分類は、学ぶ順（道具 → HTML・CSS → TypeScript・React → Python・Django → Rust・Topcoat → SQL・メール・安全）です。下の「分類の一覧」から選ぶか、HTML 版（`webapp_learning\html`）で `Ctrl` + `F` を押して、やりたいことの言葉（「しぼる」「並べかえる」「送る」など）で探してください。
- **同じやりたいことでも、言語ごとに書き方が違います。** 例えば「並びから条件に合う物だけを集める」は、TypeScript では `items.filter(…)`、Python では `[r for r in rows if …]`、Django では `Request.objects.filter(…)` です。それぞれの言語の分類に載せています。
- 「**書き方**」**の列:** そのまま使える形の、短いコードです。`…` の所は、自分の場面に合わせて書きます。
- 「**ひとこと**」**の列:** 気をつけることや、結果の例です。くわしい意味は、12 付録の一覧と、本文の「▶ ここで初めて出てくるコード」の箱にあります。
- 「**出てくる所**」**の列:** その書き方が、00〜11 で初めて出てくる巻と章です（399 個）。
- 「＋ **合わせて覚える**」: 本文で使った書き方と一緒に覚えておくと、すぐ役に立つ書き方です（44 個）。この付録で足しました。使う前に、12 付録の一覧や、それぞれの説明書で、くわしい使い方を確かめてください。

> **ヒント:** 練習アプリを作っている時に「こういう時はどう書くんだっけ？」と思ったら、まずこの付録を開いてください。1 つの分類の表を上から下まで一度ながめておくと、あとで探す時に思い出しやすくなります。

分類の一覧

分類	数	分類	数
道具を入れる・確かめる	18	Python の文字・数・並び・辞書・ファイル・日時	47

分類	数	分類	数
npm（Node.js の部品）	15	Django（モデル・ORM）	31
HTML（画面の中身）	41	Django（URL・ビュー・テンプレート・管理画面・設定）	35
CSS（画面の見た目）	26	Django REST framework（API）	11
TypeScript の基本	35	Rust と Topcoat（練習アプリの Rust 版）	24
TypeScript でよく使う道具（並び・文字・数・日付・JSON・通信）	44	SQL と PostgreSQL	19
React（画面を部品で作る）	27	メールを送る（SMTP）	5
Python の道具（venv・pip・manage.py）	18	安全とテスト	11
Python の基本	36		

道具を入れる・確かめる

練習アプリを作るのに使う道具の、版の確かめ方と、よく使うコマンドです。

やりたいこと	書き方	ひとこと	出てくる所
VS Code の版を見る	`code --version`		00 6. 練習の環境を用意する
Node.js と npm の版を見る	`node --version` `npm --version`		00 6. 練習の環境を用意する
Python の版を見る	`py --version`	Windows では py が起動係	00 6. 練習の環境を用意する
Rust の版を見る	`rustc --version` `cargo --version`		07 1. 準備
今いるフォルダを知る・移る	`pwd` `cd webapp_learning\samples`		00 6. 練習の環境を用意する

やりたいこと	書き方	ひとこと	出てくる所
フォルダの中を見る	`ls` `Get-ChildItem`		＋ 合わせて覚える
フォルダを作る	`mkdir samples`		01 2. HTML の基本
ファイルの中を見る	`Get-Content dev.env`		06 前編 2. 開発の準備
環境変数を入れる（この窓だけ）	`$env:DJANGO_DEBUG = "true"`		06 後編 1. 開発中と、ビルドした後
今いるフォルダのプログラムを動かす	`.\.venv\Scripts\python.exe manage.py runserver`	`.\` が要る	04 1. 準備: 仮想環境を作って、部品を入れる
最後のプログラムが成功したか	`$LASTEXITCODE`	0 なら成功	06 前編 2. 開発の準備
1 行に 2 つのコマンド	`$env:SMTP_HOST = "127.0.0.1";` `$env:SMTP_PORT = "1025"`		09 8. 手元で、アプリから送ってみる
長いコマンドを次の行へ続ける（行の最後に書く）	`` ` ``		06 後編 1. 開発中と、ビルドした後
スクリプトの引数を決める	`param([string[]] $ManageArgs)`		06 前編 2. 開発の準備
エラーで止めるスクリプトにする	`$ErrorActionPreference = "Stop"`		06 前編 2. 開発の準備
スクリプトの隣のファイルを指す	`Join-Path $PSScriptRoot "dev.env"`		06 前編 2. 開発の準備
引数をまとめて渡す	`& $python manage.py @ManageArgs`		06 前編 2. 開発の準備
行が形に合うか調べる	`if ($line -match '^\s*#') { continue }`		06 前編 2. 開発の準備

npm（Node.js の部品）

TypeScript と React の部品は npm で入れます。`package.json` があるフォルダで打ちます。

やりたいこと	書き方	ひとこと	出てくる所
package.json を作る	`npm init -y`		＋ 合わせて覚える
部品を入れる	`npm install`	package.json の部品を全部	02 1. TypeScript とは
部品を足す	`npm install react`	dependencies に入る	02 1. TypeScript とは
開発の時だけの部品を足す	`npm install -D typescript`	devDependencies に入る	＋ 合わせて覚える
Cargo.lock のように版をそろえて入れる	`npm ci`	package-lock.json のとおり	＋ 合わせて覚える
登録したコマンドを動かす	`npm run check`	"scripts" に書いた物	03 1. 動かしてみる
開発用のサーバーを動かす	`npm run dev`	Vite が動く	03 1. 動かしてみる
本番用に作る	`npm run build`	dist フォルダにできる	03 1. 動かしてみる
型を確かめる	`npx tsc`		02 1. TypeScript とは
TypeScript を直接動かす	`node src/main.ts`	新しい Node.js は .ts を読める	00 6. 練習の環境を用意する
古い部品を調べる	`npm outdated`		＋ 合わせて覚える
弱点のある部品を調べる	`npm audit`		＋ 合わせて覚える
登録したコマンドの名前	`"scripts": { "check": "tsc" }`		02 1. TypeScript とは
開発の時だけの部品の一覧	`"devDependencies": { … }`		02 1. TypeScript とは

やりたいこと	書き方	ひとこと	出てくる所
型の確かめをきびしくする	`"strict": true`	tsconfig.json	02 1. TypeScript とは

HTML（画面の中身）

画面の中身と組み立てを書く HTML の、よく使うタグと属性です。

やりたいこと	書き方	ひとこと	出てくる所
ページの最初の宣言	`<!doctype html>`		01 2. HTML の基本
言葉と文字の符号	`<html lang="ja">` `<meta charset="utf-8">`		01 2. HTML の基本
スマホの幅に合わせる	`<meta name="viewport" content="width=device-width, initial-scale=1">`		01 3. フォーム: 依頼の入力画面
ページの題（タブに出る）	`<title>依頼の一覧</title>`		01 2. HTML の基本
CSS のファイルをつなぐ	`<link rel="stylesheet" href="style.css">`		01 3. フォーム: 依頼の入力画面
JavaScript を読み込む	`<script type="module" src="main.js"></script>`		08 後編 5. 入力フォーム
見出し	`<h1>依頼の一覧</h1>`	h1 はページに 1 つ	01 2. HTML の基本
段落	`<p>…</p>`		01 2. HTML の基本
リンク	`<a href="/requests">一覧へ</a>`		01 3. フォーム: 依頼の入力画面
画像	`<img src="logo.png" alt="会社のロゴ">`	alt は読み上げの説明	＋ 合わせて覚える
囲み（意味なし）	`<div class="card">…</div>`		01 3. フォーム: 依頼の入力画面
文の一部を囲む	`<span class="badge">回覧中</span>`		01 4. 表: 依頼の一覧

やりたいこと	書き方	ひとこと	出てくる所
ページの頭・主な中身・足元	`<header>` `<main>` `<footer>`		01 3. フォーム: 依頼の入力画面
メニュー	`<nav><a href="/">トップ</a></nav>`		01 3. フォーム: 依頼の入力画面
箇条書き	`<ul><li>…</li></ul>`	番号付きは	03 5. 入力フォーム
表	`<table><thead><tr><th>件名</th></tr></thead><tbody>…</tbody></table>`		01 4. 表: 依頼の一覧
表のマスを横につなぐ	`<td colspan="4">`		05 7. HTML の画面
送信フォーム	`<form action="/requests" method="post">`		01 3. フォーム: 依頼の入力画面
ファイルを送るフォーム	`<form method="post" enctype="multipart/form-data">`		01 3. フォーム: 依頼の入力画面
入力欄	`<input id="title" name="title" type="text">`		01 3. フォーム: 依頼の入力画面
入力欄の説明（名前）	`<label for="title">件名</label>`	押すと欄に入れる	01 3. フォーム: 依頼の入力画面
必ず入れる欄	`<input required>`		01 3. フォーム: 依頼の入力画面
文字数の上限	`<input maxlength="100">`		01 3. フォーム: 依頼の入力画面
空の時の例の文字	`<input placeholder="例：治具の製作">`		01 3. フォーム: 依頼の入力画面
数の欄と範囲	`<input type="number" min="1" max="99">`		11 B. サンプル集
日付の欄	`<input type="date">`		01 3. フォーム: 依頼の入力画面
チェックボックス	`<input type="checkbox" checked>`		11 B. サンプル集

やりたいこと	書き方	ひとこと	出てくる所
ファイルを選ぶ欄	`<input type="file" accept=".jpg,.png,.pdf" multiple>`		01 3. フォーム: 依頼の入力画面
何行も書ける欄	`<textarea rows="4"></textarea>`		01 3. フォーム: 依頼の入力画面
選ぶ欄	`<select name="kind"><option value="a">A</option></select>`		01 3. フォーム: 依頼の入力画面
最初から選んでおく	`<option selected>`		11 B. サンプル集
ボタン	`<button type="submit">送る</button>`		01 3. フォーム: 依頼の入力画面
読み上げ用の名前	`aria-label="回覧者1"`		01 3. フォーム: 依頼の入力画面
大事なお知らせを読み上げる	`role="alert"`		08 前編 7. 1 回限りの承認の URL
見た目のための名前	`class="note"`		01 2. HTML の基本
ページで 1 つだけの名前	`id="title"`		01 3. フォーム: 依頼の入力画面
自分で決める属性	`data-id="42"`	JavaScript から el.dataset.id	＋ 合わせて覚える
特別な文字を書く	`<` `>` `&`	< > & を文字として	10 6. XSS: 画面に、スクリプトを動かされる
コメント	`<!-- メモ -->`		11 B. サンプル集
絵を線で描く	`<svg viewBox="0 0 400 300"><rect x="10" y="10" width="50" height="30" /></svg>`		06 前編 6. 1 回限りの承認の URL
開いたり閉じたりする部分	`<details><summary>見出し</summary>…</details>`		＋ 合わせて覚える

CSS（画面の見た目）

見た目を決める CSS の、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所
クラスで選ぶ	`.note { … }`	class="note" の物全部	01 7. 並べ方: flex と grid
id で選ぶ	`#title { … }`		＋ 合わせて覚える
中にある物を選ぶ	`.form-grid label { … }`		11 B. サンプル集
いくつかに同じ見た目	`h1, h2 { … }`		＋ 合わせて覚える
全部のタグ	`* { box-sizing: border-box; }`		11 B. サンプル集
マウスを乗せた時	`button:hover { … }`		11 B. サンプル集
キーボードで選ばれた時	`button:focus-visible { outline: 2px solid; }`		＋ 合わせて覚える
色を変数にする	`:root { --navy: #1e3a5f; }` `color: var(--navy);`		01 5. CSS の基本
色を書く	`#1e3a5f` `rgb(30 58 95)`		01 5. CSS の基本
文字の色・背景の色	`color: #0f172a; background: #f8fafc;`		01 5. CSS の基本
文字の大きさ・太さ	`font-size: 1.25rem; font-weight: 700;`		11 B. サンプル集
行の高さ	`line-height: 1.6;`		＋ 合わせて覚える
内側の余白・外側の余白	`padding: 8px 12px; margin: 0 auto;`		01 8. 画面の幅に合わせる
枠線と角の丸み	`border: 1px solid #cbd5e1;` `border-radius: 6px;`		11 B. サンプル集
幅・高さ	`width: 100%; max-width: 960px;`		01 8. 画面の幅に合わせる
横に並べる	`display: flex; gap: 8px; align-items: center;`		01 7. 並べ方: flex と grid
両端に寄せて並べる	`justify-content: space-between;`		01 7. 並べ方: flex と grid

やりたいこと	書き方	ひとこと	出てくる所
格子に並べる	`display: grid; grid-template-columns: 8em 1fr;`		01 7. 並べ方: flex と grid
残りの場所を分ける	`1fr`		01 7. 並べ方: flex と grid
列がはみ出さないようにする	`minmax(0, 1fr)`		01 8. 画面の幅に合わせる
画面が狭い時だけ	`@media (max-width: 640px) { … }`		01 8. 画面の幅に合わせる

大きさの単位	`16px`
	`1rem`
	`50%`
		rem は文字の大きさの何倍か	01 7. 並べ方: flex と grid

隠す	`display: none;`		01 7. 並べ方: flex と grid
影を付ける	`box-shadow: 0 1px 3px rgb(0 0 0 / 0.2);`		＋ 合わせて覚える
文字を中央に	`text-align: center;`		11 B. サンプル集
コメント	`/* メモ */`		01 7. 並べ方: flex と grid

TypeScript の基本

型を書いて、間違いを動かす前に見つける JavaScript です。

やりたいこと	書き方	ひとこと	出てくる所
変数を作る（変えない）	`const name = "山田";`	ふつうはこれ	02 2. 変数・配列・オブジェクト・関数
変数を作る（あとで変える）	`let count = 0;`		02 2. 変数・配列・オブジェクト・関数
型を書く	`const isOpen: boolean = true;`		02 2. 変数・配列・オブジェクト・関数

やりたいこと	書き方	ひとこと	出てくる所
文字・数の型	`string` `number`		02 2. 変数・配列・オブジェクト・関数
並びの型	`string[]`		02 2. 変数・配列・オブジェクト・関数
オブジェクトの形に名前を付ける	`type Request = { id: number; title: string };`		02 3. 依頼の状態を型にする
オブジェクトの形（interface）	`interface User { name: string }`		11 B. サンプル集
無くてもよい項目	`note?: string;`		02 3. 依頼の状態を型にする
決まった値のどれか	`type Status = "下書き" | "回覧中" | "承認済み";`		02 3. 依頼の状態を型にする
値か null	`string | null`		02 3. 依頼の状態を型にする
キーと値の表の型	`Record<string, number>`		02 5. 図面を JSON で保存する
変えない決まった値にする	`["a", "b"] as const`		11 B. サンプル集
型だけ読み込む	`import type { Status } from "./types.ts";`		02 3. 依頼の状態を型にする
関数を作る	`function add(a: number, b: number): number { return a + b; }`		02 2. 変数・配列・オブジェクト・関数
短い関数（アロー関数）	`const double = (x: number) => x * 2;`		02 2. 変数・配列・オブジェクト・関数
外で使えるようにする	`export function label(s: Status)`		02 3. 依頼の状態を型にする

やりたいこと	書き方	ひとこと	出てくる所
読み込む	`import { label } from "./status.ts";`		02 4. 配列の道具: 絞る・変える・まとめる
条件で分ける	`if (a === b) { … } else { … }`	=== は型も比べる	02 3. 依頼の状態を型にする
条件で値を選ぶ	`user.isAdmin ? "管理者" : "一般"`		02 2. 変数・配列・オブジェクト・関数

かつ・または・ではない	`a && b`
	`a || b`
	`!a`
			02 5. 図面を JSON で保存する

null か undefined なら別の値	`name ?? "（なし）"`		02 4. 配列の道具: 絞る・変える・まとめる
あれば中を見る	`user?.email`	無ければ undefined	08 後編 6. 図面
null ではないと言い切る	`document.getElementById("root")!`	確かな時だけ	03 7. 画面の切りかえと、部品の組み立て
形で分ける	`switch (status) { case "下書き": … break; }`		02 3. 依頼の状態を型にする
くり返す	`for (const r of requests) { … }`		02 4. 配列の道具: 絞る・変える・まとめる
文字の中に値を入れる	`` `${r.title} のメモ` ``		02 2. 変数・配列・オブジェクト・関数
オブジェクトから取り出す	`const { name, email } = user;`		11 B. サンプル集
並びから取り出す	`const [first, second] = items;`		03 3. props と state
中身を広げてコピー	`{ ...drawing, shapes: [...drawing.shapes, s] }`		02 4. 配列の道具: 絞る・変える・まとめる

やりたいこと	書き方	ひとこと	出てくる所
型を確かめて分ける	`typeof v === "number"`		02 5. 図面を JSON で保存する
エラーを起こす・受け止める	`throw new Error("…");` `try { … } catch (err) { … }`		02 7. サーバーとのやり取り
エラーの種類を調べる	`err instanceof Error ? err.message : "…"`		06 後編 3. サーバーとの窓口
クラスを作る	`class Counter { #count = 0; add() { this.#count++; } }`		06 後編 3. サーバーとの窓口
1 足す	`n++` `n += 1`		＋ 合わせて覚える
コメント	`// メモ` `/* メモ */`		02 2. 変数・配列・オブジェクト・関数

TypeScript でよく使う道具（並び・文字・数・日付・JSON・通信）

配列（並び）・文字・数・日付・JSON の、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所
数を数える	`items.length`		02 2. 変数・配列・オブジェクト・関数
1 つずつ変えた並びを作る	`items.map((r) => r.title)`		02 4. 配列の道具: 絞る・変える・まとめる
条件に合う物だけの並び	`items.filter((r) => r.status === "回覧中")`		02 4. 配列の道具: 絞る・変える・まとめる
最初に合う物	`items.find((r) => r.id === id)`		11 B. サンプル集
合う物があるか・全部合うか	`items.some(…)` `items.every(…)`		11 B. サンプル集
1 つにまとめる	`items.reduce((sum, r) => sum + r.amount, 0)`		11 B. サンプル集

やりたいこと	書き方	ひとこと	出てくる所
並べかえる	`[...items].sort((a, b) => a.id - b.id)`	sort は元を変えるのでコピーしてから	02 4. 配列の道具: 絞る・変える・まとめる
含むか	`ids.includes(3)`		03 4. 一覧の画面: 読み込みと、しぼり込み
後ろに足す	`list.push(x)`		03 5. 入力フォーム
区切りでつなぐ	`approvers.join("、")`		02 2. 変数・配列・オブジェクト・関数
1 つずつ処理する	`items.forEach((r) => console.log(r))`		11 B. サンプル集
何番目か	`items.findIndex(…)`		11 B. サンプル集
並びか調べる	`Array.isArray(v)`		02 5. 図面を JSON で保存する
重なりを消す	`[...new Set(ids)]`		03 5. 入力フォーム

キーと値の入れ物	`new Map()`
	`.set(k, v)`
	`.get(k)`
			11 B. サンプル集

オブジェクトのキー・値	`Object.keys(o)`
	`Object.values(o)`
	`Object.entries(o)`
			06 後編 3. サーバーとの窓口

前後の空白を取る	`s.trim()`		03 5. 入力フォーム
区切りで分ける	`s.split(",")`		11 B. サンプル集
含むか・始まるか	`s.includes("治具")` `s.startsWith("#/")`		03 4. 一覧の画面: 読み込みと、しぼり込み
小文字にする	`s.toLowerCase()`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
置きかえる	`s.replace(/\s+/g, " ")`	g で全部	04 6. 日本時間と、承認の URL の文字列
形に合うか（正規表現）	`/^\d{6}$/.test(no)` `hash.match(/(\d+)/)`		03 4. 一覧の画面: 読み込みと、しぼり込み
前を 0 で埋める	`String(n).padStart(3, "0")`		11 B. サンプル集
文字を数にする	`Number("42")` `parseInt("42", 10)`		06 後編 4. ログインと、画面の切りかえ
小数のけたを決める	`x.toFixed(2)`		11 B. サンプル集
数を 3 けたで区切る	`n.toLocaleString("ja-JP")` `new Intl.NumberFormat("ja-JP").format(n)`		11 B. サンプル集
四捨五入・大きい方	`Math.round(x)` `Math.max(a, b)`		03 6. 図面の画面
日時を作る	`new Date("2026-11-05T09:00:00+09:00")`		06 後編 6. 依頼の詳細と、操作のボタン
日時を日本の形の文字にする	`d.toLocaleString("ja-JP", { dateStyle: "short", timeStyle: "short" })`		06 後編 6. 依頼の詳細と、操作のボタン
JSON の文字にする	`JSON.stringify(obj)`		02 5. 図面を JSON で保存する
JSON の文字から読む	`JSON.parse(text)`	壊れていると SyntaxError	02 5. 図面を JSON で保存する
サーバーに要求を送る	`const res = await fetch("/api/requests");`		02 7. サーバーとのやり取り
答えがうまくいったか	`if (!res.ok) throw new Error(`エラー ${res.status}`);`		02 7. サーバーとのやり取り

やりたいこと	書き方	ひとこと	出てくる所
答えの JSON を読む	`const data = await res.json();`		05 9. 自動のテスト
POST で JSON を送る	`fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(x) })`		02 7. サーバーとのやり取り
ファイルを送る	`const fd = new FormData(); fd.append("file", f); fetch(url, { method: "POST", body: fd })`		06 後編 3. サーバーとの窓口
あとで終わる処理を待つ	`await sleep(100)`	async の関数の中で	02 7. サーバーとのやり取り
async の関数を作る	`async function load() { … }`		02 7. サーバーとのやり取り
決めた時間の後に動かす	`const t = setTimeout(() => …, 3000); clearTimeout(t);`		06 後編 5. 依頼を出す
URL の %XX を戻す	`decodeURIComponent(s)`	反対は encodeURIComponent	06 後編 3. サーバーとの窓口
クッキーを読む	`document.cookie`		06 後編 3. サーバーとの窓口
タグを探す	`document.querySelector("#title")`		＋ 合わせて覚える
タグの中の文字を変える	`el.textContent = "…"`	安全（HTML として動かない）	08 後編 6. 図面
画面に出す（調べ物）	`console.log(x)`	F12 の Console に出る	02 2. 変数・配列・オブジェクト・関数

React（画面を部品で作る）

画面を部品（コンポーネント）に分けて作る React の、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所
部品を作る	`function StatusBadge({ status }: { status: Status }) { return <span>{status}</span>; }`	名前は大文字で始める	02 2. 変数・配列・オブジェクト・関数
部品に値を渡す（props）	`<StatusBadge status={r.status} />`		＋ 合わせて覚える
JSX に値を入れる	`<h1>{r.title}</h1>`	{ } の中は式	＋ 合わせて覚える
見た目の名前	`className="btn-primary"`	class ではない	03 2. 部品
入力欄の説明	`<label htmlFor="title">`	for ではない	03 5. 入力フォーム
状態を持つ	`const [count, setCount] = useState(0);`		03 3. props と state
状態を変える	`setCount(count + 1)`	変えると描き直す	03 3. props と state
前の値から変える	`setItems((prev) => [...prev, item])`		＋ 合わせて覚える
入力欄と状態を結ぶ	`<input value={title} onChange={(e) => setTitle(e.target.value)} />`		03 4. 一覧の画面: 読み込みと、しぼり込み
チェックと状態を結ぶ	`<input type="checkbox" checked={on} onChange={(e) => setOn(e.target.checked)} />`		11 B. サンプル集
押した時	`<button onClick={() => setOpen(!open)}>`		03 6. 図面の画面
フォームを送った時	`<form onSubmit={handleSubmit}> e.preventDefault()`		03 5. 入力フォーム
送信中は押せなくする	`<button disabled={sending}>`		03 5. 入力フォーム
並びを表示する	`{items.map((r) => <tr key={r.id}>…</tr>)}`	key を付ける	03 4. 一覧の画面: 読み込みと、しぼり込み
条件が合う時だけ表示	`{errors.length > 0 && <ul>…</ul>}`		03 5. 入力フォーム

やりたいこと	書き方	ひとこと	出てくる所
どちらかを表示	`{loading ? <p>読み込み中</p> : <List />}`		02 2. 変数・配列・オブジェクト・関数
余分なタグ無しで包む	`<> … </>`		03 7. 画面の切りかえと、部品の組み立て
画面が出た後に読み込む	`useEffect(() => { load(); }, []);`	[] で 1 回だけ	03 4. 一覧の画面: 読み込みと、しぼり込み
後片づけ	`useEffect(() => { const t = setTimeout(…); return () => clearTimeout(t); }, [x]);`		03 4. 一覧の画面: 読み込みと、しぼり込み
関数を覚えておく	`const loadMe = useCallback(async () => { … }, []);`		06 後編 4. ログインと、画面の切りかえ
タグを覚えておく	`const ref = useRef<HTMLInputElement>(null); ref.current?.focus()`		02 2. 変数・配列・オブジェクト・関数
計算を覚えておく	`useMemo(() => heavy(items), [items])`		＋ 合わせて覚える
画面を描き始める	`createRoot(document.getElementById("root")!).render(<App />)`		03 7. 画面の切りかえと、部品の組み立て
入力の型	`(e: React.ChangeEvent<HTMLInputElement>)`		＋ 合わせて覚える
送信の型	`(e: FormEvent)`		＋ 合わせて覚える
JSX のコメント	`{/* メモ */}`		11 B. サンプル集
Vite の設定	`export default defineConfig({ plugins: [react()] })`		06 後編 1. 開発中と、ビルドした後

Python の道具（venv・pip・manage.py）

Python の部品を、プロジェクトごとの入れ物（仮想環境 `.venv`）に入れて使います。

やりたいこと	書き方	ひとこと	出てくる所
仮想環境を作る	`py -m venv .venv`	.venv フォルダができる	04 1. 準備: 仮想環境を作って、部品を入れる
仮想環境の python で部品を入れる	`.\.venv\Scripts\python.exe -m pip install -r requirements.txt`		04 1. 準備: 仮想環境を作って、部品を入れる
部品を 1 つ足す	`.\.venv\Scripts\python.exe -m pip install django`		04 1. 準備: 仮想環境を作って、部品を入れる
入っている部品の一覧	`python -m pip list`		04 1. 準備: 仮想環境を作って、部品を入れる
入っている部品を書き出す	`python -m pip freeze > requirements.txt`		04 1. 準備: 仮想環境を作って、部品を入れる
Python の短いプログラムをその場で動かす	`python -c "print(1 + 2)"`		05 6. ORM: Python でデータベースを操作する
Django のプロジェクトを作る	`django-admin startproject config .`		05 2. プロジェクトとアプリを作る
アプリ（機能のまとまり）を作る	`python manage.py startapp workflow`		05 2. プロジェクトとアプリを作る
開発用のサーバーを動かす	`python manage.py runserver`	http://127.0.0.1:8000/	05 5. 管理画面
表の変更を記録する	`python manage.py makemigrations`		05 4. マイグレーション: テーブルを作る

やりたいこと	書き方	ひとこと	出てくる所
表を作る・変える	`python manage.py migrate`		05 4. マイグレーション: テーブルを作る
管理者を作る	`python manage.py createsuperuser`		05 5. 管理画面
Python の対話の画面（Django の設定つき）	`python manage.py shell`		05 6. ORM: Python でデータベースを操作する
テストを動かす	`python manage.py test workflow`		05 9. 自動のテスト
設定の間違いを調べる	`python manage.py check --deploy`	本番の前に	＋ 合わせて覚える
静的ファイルを集める	`python manage.py collectstatic --noinput`		06 後編 1. 開発中と、ビルドした後
自分のコマンドを動かす	`python manage.py import_employees employees.csv`		06 前編 1. 全体の形
本番用のサーバーで動かす	`waitress-serve --listen=127.0.0.1:8081 config.wsgi:application`		06 前編 2. 開発の準備

Python の基本

字下げ（空白 4 つ）で、まとまりを表します。

やりたいこと	書き方	ひとこと	出てくる所
変数に入れる	`name = "山田"`	型は書かなくてよい	＋ 合わせて覚える
型の説明を付ける	`app_name: str = "依頼・承認アプリ"`		04 2. Python の書き方
関数を作る	`def add(a: int, b: int) -> int: return a + b`		04 2. Python の書き方
引数の初めの値	`def env(name, default=None):`		04 2. Python の書き方

やりたいこと	書き方	ひとこと	出てくる所
名前を付けて渡す	`open(path, encoding="utf-8")`		04 2. Python の書き方
いくつでも受け取る	`def f(*args, **kwargs):`		11 B. サンプル集
辞書をばらして渡す	`create_user(email, **extra)`		06 前編 4. モデル
戻り値の型の説明	`-> None` `-> str | None`		04 2. Python の書き方
条件で分ける	`if x > 0: … elif x == 0: … else: …`		04 2. Python の書き方
条件で値を選ぶ	`a if 条件 else b`		06 前編 7. 図面と添付ファイルの確かめ

かつ・または・ではない	`a and b`
	`a or b`
	`not a`
			06 前編 5. 状態を変える処理を 1 か所に集める

含むか・含まないか	`x in items` `key not in d`		04 3. 依頼の状態と、状態の移り変わり
None か調べる	`x is None` `x is not None`		09 2. SMTP の会話を見る
くり返す	`for r in rows:`		04 2. Python の書き方
番号付きでくり返す	`for i, r in enumerate(rows, start=1):`		04 2. Python の書き方
決まった回数くり返す	`for i in range(3):`		＋ 合わせて覚える
2 つを組にしてくり返す	`for a, b in zip(xs, ys):`		11 B. サンプル集
条件の間くり返す	`while line := f.readline():`	:= は入れて使う	09 2. SMTP の会話を見る
抜ける・次へ	`break` `continue`		09 2. SMTP の会話を見る

やりたいこと	書き方	ひとこと	出てくる所
何もしない	`pass`		09 2. SMTP の会話を見る
エラーを受け止める	`try: … except ValueError as e: …`		04 3. 依頼の状態と、状態の移り変わり
エラーを起こす	`raise RuntimeError("…")`		04 3. 依頼の状態と、状態の移り変わり
自分のエラーの種類	`class TransitionError(Exception): pass`		04 3. 依頼の状態と、状態の移り変わり
後片づけを自動で	`with open(path, encoding="utf-8") as f:`		04 5. 社員の CSV を読み込む
クラスを作る	`class Item: def __init__(self, title): self.title = title`		11 B. サンプル集
表示の形を決める	`def __str__(self) -> str:`		05 3. モデル: データの設計図
項目だけ書くクラス	`@dataclass class Row: no: str; name: str`		04 3. 依頼の状態と、状態の移り変わり
決まった値の一覧	`class Status(StrEnum): DRAFT = "下書き"`		04 3. 依頼の状態と、状態の移り変わり
直接動かした時だけ	`if __name__ == "__main__":`		04 3. 依頼の状態と、状態の移り変わり
このファイルの場所	`Path(__file__).resolve().parent`		06 前編 3. 設定
読み込む	`from pathlib import Path import json`		04 3. 依頼の状態と、状態の移り変わり
名前の無い短い関数	`lambda r: r.id`		06 前編 5. 状態を変える処理を 1 か所に集める

やりたいこと	書き方	ひとこと	出てくる所

割り算・切り捨て・余り・べき乗	`7 / 2`
	`7 // 2`
	`7 % 2`
	`2 ** 10`
			11 B. サンプル集

足して入れる	`total += n`		06 前編 5. 状態を変える処理を 1 か所に集める
コメント	`# メモ`		04 2. Python の書き方
説明の文（docstring）	`"""この関数は…"""`		04 3. 依頼の状態と、状態の移り変わり

Python の文字・数・並び・辞書・ファイル・日時

よく使う型の道具です。

やりたいこと	書き方	ひとこと	出てくる所
文字の中に値を入れる	`f"{name} さん"`		04 2. Python の書き方
小数 2 けたで表示	`f"{x:.2f}"`		11 B. サンプル集
3 けた区切り	`f"{amount:,}"`		11 B. サンプル集
前を 0 で埋める	`f"{n:05d}"`		11 B. サンプル集
幅をそろえる	`f"{name:<16}"` `f"{n:>6}"`		11 B. サンプル集
割合で表示	`f"{0.256:.1%}"`	25.6%	11 B. サンプル集
' 付きで表示（調べ物）	`f"{no!r}"`		04 5. 社員の CSV を読み込む
名前と値を一緒に表示	`f"{x=}"`	x=42	＋ 合わせて覚える
日時を形を決めて表示	`f"{now:%Y-%m-%d %H:%M}"`		06 前編 5. 状態を変える処理を 1 か所に集める

やりたいこと	書き方	ひとこと	出てくる所
`\` をそのまま書く	`r"\d{6}"`		04 5. 社員の CSV を読み込む
前後の空白を取る	`s.strip()`		04 5. 社員の CSV を読み込む
区切りで分ける	`s.split(",")`		06 前編 3. 設定
後ろから 1 回だけ分ける	`name.rsplit(".", 1)`		06 前編 7. 図面と添付ファイルの確かめ
並びをつなぐ	`"、".join(names)`		09 2. SMTP の会話を見る
置きかえる	`s.replace("\r\n", "\n")`		04 6. 日本時間と、承認の URL の文字列
始まり・終わりを調べる	`s.startswith("http")` `s.endswith(".pdf")`		06 前編 7. 図面と添付ファイルの確かめ
小文字・大文字	`s.lower()` `s.upper()`		06 前編 3. 設定
数字だけか	`s.isdigit()`		＋ 合わせて覚える
文字を数にする	`int("42")` `float("1.5")`	数でないと ValueError	06 前編 3. 設定
長さ	`len(items)`		04 2. Python の書き方
並びを作る・足す	`items = []` `items.append(x)`		04 2. Python の書き方
一部を取り出す	`items[1:3]` `items[-1]`		04 5. 社員の CSV を読み込む
並べかえる	`sorted(rows, key=lambda r: r.id)`		11 B. サンプル集
1 つずつ変えた並び	`[r.title for r in rows if r.open]`		04 2. Python の書き方

やりたいこと	書き方	ひとこと	出てくる所
辞書を作る	`{u.pk: u for u in users}`		05 6. ORM: Python でデータベースを操作する
辞書から取り出す	`d.get("name", "（なし）")`	無ければ 2 つ目	04 2. Python の書き方
辞書の全部	`for k, v in d.items():`		11 B. サンプル集

合計・最大・最小	`sum(xs)`
	`max(xs)`
	`min(xs)`
			11 B. サンプル集

どれか・全部	`any(…)` `all(…)`		06 前編 7. 図面と添付ファイルの確かめ
数を数える	`Counter(words).most_common(3)`		11 B. サンプル集
無いキーに初めの値	`defaultdict(list)`		11 B. サンプル集
重なりを消す	`set(items)`		04 5. 社員の CSV を読み込む
ファイルを読む	`Path("a.txt").read_text(encoding="utf-8")`		11 B. サンプル集
ファイルに書く	`Path("a.txt").write_text(s, encoding="utf-8")`		11 B. サンプル集
あるか調べる	`Path("a.txt").exists()`		06 前編 3. 設定
場所をつなぐ	`BASE_DIR / "media"`		04 5. 社員の CSV を読み込む

名前と拡張子	`p.name`
	`p.suffix`
	`p.stem`
			11 B. サンプル集

CSV を読む	`csv.DictReader(f)`	1 行が辞書になる	04 5. 社員の CSV を読み込む
JSON の文字にする・読む	`json.dumps(d, ensure_ascii=False)` `json.loads(s)`		06 前編 7. 図面と添付ファイルの確かめ

やりたいこと	書き方	ひとこと	出てくる所
環境変数を読む	`os.environ.get("DJANGO_DEBUG", "false")`		06 前編 3. 設定
引数を受け取る	`sys.argv[1:]`		04 5. 社員の CSV を読み込む
今の日時（日本）	`datetime.now(ZoneInfo("Asia/Tokyo"))`		04 6. 日本時間と、承認の URL の文字列
日時を文字にする	`d.strftime("%Y-%m-%d %H:%M")`		04 6. 日本時間と、承認の URL の文字列
日時の差	`timedelta(days=7)` `(b - a).days`		04 6. 日本時間と、承認の URL の文字列
推測できない文字を作る	`secrets.token_urlsafe(32)`		04 6. 日本時間と、承認の URL の文字列
形に合うか（正規表現）	`re.fullmatch(r"\d{6}", no)`		04 5. 社員の CSV を読み込む
年月日の書き方	`%Y` `%m` `%d` `%H` `%M` `%S`		04 6. 日本時間と、承認の URL の文字列

Django（モデル・ORM）

データベースの表を Python のクラス（モデル）で書き、ORM で読み書きします。

やりたいこと	書き方	ひとこと	出てくる所
表を作る（モデル）	`class Request(models.Model):` `title = models.CharField(max_length=100)`		05 3. モデル: データの設計図
文字の列	`models.CharField(max_length=100)`		05 3. モデル: データの設計図

やりたいこと	書き方	ひとこと	出てくる所
長い文字の列	`models.TextField(blank=True)`		05 3. モデル: データの設計図
数の列	`models.IntegerField(default=0)`		＋ 合わせて覚える
日時の列	`models.DateTimeField(auto_now_add=True)`	作った時に自動で入る	05 3. モデル: データの設計図
ほかの表を指す	`models.ForeignKey(User, on_delete=models.PROTECT)`		05 3. モデル: データの設計図
選び肢	`class Status(models.TextChoices): DRAFT = "draft", "下書き"`		＋ 合わせて覚える
選び肢を使う列	`models.CharField(choices=Status.choices)`		05 3. モデル: データの設計図
並び順や表の名前	`class Meta: ordering = ["-created_at"]`	- は新しい順	05 3. モデル: データの設計図
全部	`Request.objects.all()`		05 6. ORM: Python でデータベースを操作する
しぼる	`Request.objects.filter(status="回覧中")`		05 6. ORM: Python でデータベースを操作する
除く	`.exclude(status="完了")`		06 前編 8. React の画面が呼ぶ API
1 つだけ	`Request.objects.get(pk=1)`	無いと DoesNotExist	04 2. Python の書き方
無ければ 404	`get_object_or_404(Request, pk=pk)`		06 前編 8. React の画面が呼ぶ API
最初の 1 つ	`.first()`	無ければ None	06 前編 5. 状態を変える処理を 1 か所に集める
あるか	`.exists()`		06 前編 3. 設定

やりたいこと	書き方	ひとこと	出てくる所
数える	`.count()`		05 6. ORM: Python でデータベースを操作する
並べる	`.order_by("-created_at")`		05 6. ORM: Python でデータベースを操作する
作る	`Request.objects.create(title="…")`		05 6. ORM: Python でデータベースを操作する
保存する	`r.save()`		05 6. ORM: Python でデータベースを操作する
まとめて変える	`.update(status="完了")`		06 前編 4. モデル
消す	`r.delete()`		05 6. ORM: Python でデータベースを操作する
以上・含む・どれか	`created_at__gte=d` `title__icontains="治具"` `pk__in=ids`		05 6. ORM: Python でデータベースを操作する
つながった表で しぼる	`filter(requester__dept="製造")`		05 6. ORM: Python でデータベースを操作する
または・かつ	`Q(a=1) | Q(b=2)`		06 前編 8. React の画面が呼ぶ API
列の値を使う	`F("round") + 1`	同時に直してもずれない	11 B. サンプル集
グループごとに数える	`.values("status").annotate(n=Count("id"))`		05 6. ORM: Python でデータベースを操作する
つながった表をまとめて読む	`.select_related("requester")`	問い合わせが減る	05 6. ORM: Python でデータベースを操作する

やりたいこと	書き方	ひとこと	出てくる所
まとめて成功・失敗にする	`with transaction.atomic():`		06 前編 9. 社員の CSV の取り込み
同時に変えないよう鍵をかける	`.select_for_update()`		06 前編 5. 状態を変える処理を 1 か所に集める
入力の確かめで失敗させる	`raise ValidationError("…")`		06 前編 7. 図面と添付ファイルの確かめ

Django（URL・ビュー・テンプレート・管理画面・設定）

来た要求に答える関数（ビュー）と、画面の型紙（テンプレート）です。

やりたいこと	書き方	ひとこと	出てくる所
URL と関数を結ぶ	`path("requests/", views.request_list)`		05 7. HTML の画面
URL から数を受け取る	`path("api/requests/<int:pk>/", views.detail)`		06 前編 8. React の画面が呼ぶ API
形で URL を受ける	`re_path(r"^(?!api/).*$", views.index)`		06 後編 3. サーバーとの窓口
ほかの urls をつなぐ	`path("api/", include(router.urls))`		05 7. HTML の画面
要求の種類	`request.method == "POST"`		06 前編 6. 1 回限りの承認の URL
フォームの値	`request.POST.get("decision")`		06 前編 6. 1 回限りの承認の URL
URL の ?mine=1	`request.GET.get("mine")`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
送られたファイル	`request.FILES.get("file")`		06 前編 8. React の画面が呼ぶ API
ログインしている人	`request.user` `request.user.is_authenticated`		05 7. HTML の画面
画面を返す	`render(request, "workflow/list.html", {"items": items})`		05 7. HTML の画面
ほかの URL へ移す	`redirect("request_detail", pk=r.pk)`		＋ 合わせて覚える
JSON を返す	`JsonResponse({"status": "ok"})`		06 後編 3. サーバーとの窓口
ファイルを返す	`FileResponse(f.open("rb"), as_attachment=True)`		06 前編 8. React の画面が呼ぶ API
ログインが要るページ	`@login_required`		05 7. HTML の画面
POST だけ受け付ける	`@require_POST`		＋ 合わせて覚える
CSRF のクッキーを渡す	`@ensure_csrf_cookie`		06 後編 3. サーバーとの窓口

ログイン・ログアウト	`authenticate(request, username=…, password=…)`
	`login(request, user)`
	`logout(request)`
			＋ 合わせて覚える

テンプレートに値を出す	`{{ r.title }}`	自動でエスケープ	05 7. HTML の画面
くり返す	`{% for r in items %} … {% empty %} … {% endfor %}`		05 7. HTML の画面
条件	`{% if r.note %} … {% endif %}`		06 前編 6. 1 回限りの承認の URL

やりたいこと	書き方	ひとこと	出てくる所
土台を受け継ぐ	`{% extends "workflow/base.html" %}` `{% block content %}{% endblock %}`		06 前編 6. 1 回限りの承認の URL
フォームの合言葉	`{% csrf_token %}`	POST のフォームに必ず	06 前編 6. 1 回限りの承認の URL
CSS などの URL	`{% load static %}` `{% static 'style.css' %}`		05 7. HTML の画面
日時の形	`{{ r.created_at|date:"Y-m-d H:i" }}`		05 7. HTML の画面
空の時の文字	`{{ r.note|default:"なし" }}`		05 7. HTML の画面
改行を に	`{{ r.detail|linebreaksbr }}`		06 前編 6. 1 回限りの承認の URL
URL を名前から作る	`{% url 'request_detail' r.pk %}`		＋ 合わせて覚える
管理画面に出す	`admin.site.register(Request)`		05 7. HTML の画面
管理画面の一覧の列	`class RequestAdmin(admin.ModelAdmin): list_display = ["id", "title"]`		＋ 合わせて覚える
受け付けるサイトの名前	`ALLOWED_HOSTS = ["webapp-stg.example.local"]`		＋ 合わせて覚える
CSRF で信用するサイト	`CSRF_TRUSTED_ORIGINS = ["http://…"]`		＋ 合わせて覚える
秘密の鍵	`SECRET_KEY = env("DJANGO_SECRET_KEY")`	ソースに書かない	＋ 合わせて覚える
自分のコマンドを作る	`class Command(BaseCommand): def handle(self, *args, **opts):`		06 前編 9. 社員の CSV の取り込み

やりたいこと	書き方	ひとこと	出てくる所
コマンドで画面に出す	`self.stdout.write("…")`		06 前編 9. 社員の CSV の取り込み
ログを書く	`logger = logging.getLogger(__name__); logger.info("…")`		＋ 合わせて覚える

Django REST framework（API）

JSON の API を作るための Django の部品です。

やりたいこと	書き方	ひとこと	出てくる所
モデルと JSON を変換する	`class RequestSerializer(serializers.ModelSerializer): class Meta: model = Request; fields = [...]`		＋ 合わせて覚える
入力を確かめる	`s = RequestInputSerializer(data=request.data); s.is_valid(raise_exception=True)`		06 前編 8. React の画面が呼ぶ API
確かめた値	`s.validated_data`		06 前編 8. React の画面が呼ぶ API
JSON にした値	`RequestSerializer(qs, many=True).data`		06 前編 8. React の画面が呼ぶ API
API の関数	`@api_view(["GET", "POST"])`		06 前編 8. React の画面が呼ぶ API
API の答え	`Response(data, status=201)`		06 前編 8. React の画面が呼ぶ API
権限が無い	`raise PermissionDenied("…")`	403	06 前編 8. React の画面が呼ぶ API

やりたいこと	書き方	ひとこと	出てくる所
URL を自動で作る	`router = DefaultRouter(); router.register("requests", RequestViewSet)`		05 7. HTML の画面
ファイルを受ける	`parser_classes = [MultiPartParser]`		06 前編 8. React の画面が呼ぶ API
URL の ?…	`request.query_params.get("mine")`		06 前編 8. React の画面が呼ぶ API
API のテスト	`class T(APITestCase): res = self.client.get("/api/…"); self.assertEqual(res.status_code, 403)`		05 9. 自動のテスト

Rust と Topcoat（練習アプリの Rust 版）

Rust の書き方と、Topcoat で画面と URL を作る書き方です。くわしくは `Rust/` の資料と、その逆引きにあります。

やりたいこと	書き方	ひとこと	出てくる所
変数	`let mut count = 0;`		07 2. 依頼の状態を enum で書く
関数	`fn label(self) -> &'static str`		07 2. 依頼の状態を enum で書く
構造体	`pub struct Request { pub id: i64, pub title: String }`		07 2. 依頼の状態を enum で書く
種類（enum）と場合分け	`match status { Status::Draft => …, _ => … }`		07 2. 依頼の状態を enum で書く
あるかもしれない値	`Option<&Request>` `if let Some(r) = find(&list, 1)`		07 4. 所有権と借用

やりたいこと	書き方	ひとこと	出てくる所
失敗するかもしれない処理	`Result<T, E>` `?`		07 2. 依頼の状態を enum で書く
文字に値を入れる	`format!("{} さん", name)` `println!("{name}")`		07 5. Topcoat の最小のアプリ
幅をそろえる・0 で埋める	`format!("{:<10}", s)` `format!("{:02X}", b)`		08 前編 6. 状態を変える処理
並びから探す	`list.iter().find(|r| r.id == id)`		11 B. サンプル集
JSON にできる型	`#[derive(Serialize, Deserialize)]`		08 後編 1. 画面の並び
名前の書き方を変える	`#[serde(rename_all = "camelCase")]`		08 後編 6. 図面
環境変数を読む	`std::env::var("APP_ENV")`		08 前編 3. 設定を読む
ページを作る	`#[page] async fn list(cx: &Cx) -> Result<impl View>`		07 5. Topcoat の最小のアプリ
フォームの送信を受ける	`#[page(POST)]`		07 5. Topcoat の最小のアプリ
画面を書く	`view! { <h1>"依頼の一覧"</h1> }`		07 5. Topcoat の最小のアプリ
画面の部品	`#[component] fn status_badge(status: Status) -> impl View`		08 後編 2. 画面の部品と、XSS を防ぐ仕組み
1 回の要求の中で覚える	`#[memoize(as_ref)]`		08 前編 5. ログインとセッション
SQL を送る（sqlx）	`sqlx::query_as("SELECT … WHERE id = $1").bind(id).fetch_one(&pool).await?`		08 前編 5. ログインとセッション
まとめて成功・失敗に	`let mut tx = pool.begin().await?;` `… tx.commit().await?;`		08 前編 6. 状態を変える処理

やりたいこと	書き方	ひとこと	出てくる所
Cargo.toml の部品	`[dependencies]` `sqlx = { version = "=0.9.0", features = ["postgres"] }`		07 5. Topcoat の最小のアプリ
本番用に作る	`cargo build --release --locked`		07 6. async と、会社のサーバーへの持ち込み
CSS などの束を作る	`topcoat asset bundle --release`		07 6. async と、会社のサーバーへの持ち込み
アプリのコマンドを動かす	`cargo run -- seed-demo`		08 前編 2. 開発の準備

テスト	`#[test]`
	`#[tokio::test]`
	`assert_eq!(a, b)`
			07 2. 依頼の状態を enum で書く

SQL と PostgreSQL

データベースの表を作り、読み書きする SQL です。値は、いつも `$1` などの場所に別に渡します。

やりたいこと	書き方	ひとこと	出てくる所
表を作る	`CREATE TABLE requests (id BIGSERIAL PRIMARY KEY, title TEXT NOT NULL)`		06 前編 2. 開発の準備
値の決まり	`CHECK (char_length(title) BETWEEN 1 AND 100)`		08 前編 4. 表
作った日時を自動で	`created_at TIMESTAMPTZ NOT NULL DEFAULT now()`		08 前編 4. 表
ほかの表を指す	`REFERENCES employees(id)`		08 前編 4. 表
読む	`SELECT id, title FROM requests WHERE status = $1 ORDER BY id DESC`		08 前編 5. ログインとセッション
値を安全に渡す	`WHERE employee_no = $1`	文字をつながない	08 前編 5. ログインとセッション

やりたいこと	書き方	ひとこと	出てくる所
並びのどれか	`WHERE id = ANY($1)`		08 前編 6. 状態を変える処理
入れる	`INSERT INTO requests (title) VALUES ($1) RETURNING id`		08 前編 5. ログインとセッション
ぶつかったら何もしない	`ON CONFLICT DO NOTHING`		08 前編 9. 毎朝 8:00 の通知
書きかえる	`UPDATE requests SET status = $1 WHERE id = $2`		08 前編 5. ログインとセッション
消す	`DELETE FROM sessions WHERE expires_at < now()`		08 前編 4. 表
数える	`SELECT count(*) FROM requests`		08 前編 6. 状態を変える処理
グループごとに	`GROUP BY status`		08 前編 10. 画面から呼ぶ、読むだけの SQL
2 つの表をつなぐ	`JOIN employees e ON e.id = r.requester_id`		08 前編 5. ログインとセッション
利用者（ロール）を作る	`CREATE ROLE webapp_dev_app LOGIN`		06 前編 2. 開発の準備
権限を与える	`GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA public TO …`		＋ 合わせて覚える
psql でパスワードを変える	`\password webapp_dev_app`		06 前編 2. 開発の準備
psql で文字を出す	`\echo '…'`		10 8. SQL インジェクション
SQL のコメント	`-- メモ`		06 前編 2. 開発の準備

メールを送る（SMTP）

アプリからメールを送る書き方と、確かめ方です。

やりたいこと	書き方	ひとこと	出てくる所
メールを組み立てる（Python）	`msg = EmailMessage(); msg["To"] = to; msg["Subject"] = "…"; msg.set_content(body)`		09 2. SMTP の会話を見る
メールを送る（Django）	`send_mail(subject, body, from_addr, [to])`		06 前編 5. 状態を変える処理を 1 か所に集める
SMTP で送る（Python）	`with smtplib.SMTP(host, 25) as s: s.starttls(); s.send_message(msg)`		09 2. SMTP の会話を見る
テストのメールを送る	`.\dev.ps1 send_test_mail --to 自分のアドレス`		08 前編 8. 社員の CSV と、管理のコマンド
送り先を検証用に変える	`f"（検証のメールです。本来の宛先: {to}）\n\n{body}"`		06 前編 5. 状態を変える処理を 1 か所に集める

安全とテスト

攻められても大丈夫かを確かめる書き方です。

やりたいこと	書き方	ひとこと	出てくる所
パスワードを安全に保存する	`user.set_password(pw)` `user.check_password(pw)`	ハッシュにする	06 前編 4. モデル
推測できない合言葉	`secrets.token_urlsafe(32)`		04 6. 日本時間と、承認の URL の文字列
合言葉の指紋で保存	`hashlib.sha256(token.encode()).hexdigest()`		04 6. 日本時間と、承認の URL の文字列
時間の差を出さずに比べる	`hmac.compare_digest(a, b)`		04 6. 日本時間と、承認の URL の文字列
HTML として動かないようにする	`{{ r.title }}` `el.textContent = s`	自動・安全な書き方	08 後編 6. 図面

やりたいこと	書き方	ひとこと	出てくる所
SQL を安全に	`cursor.execute("… WHERE no = %s", [no])`		08 前編 5. ログインとセッション
ファイルの中身で種類を確かめる	`data.startswith(b"\x89PNG\r\n")`		06 前編 7. 図面と添付ファイルの確かめ
テストのクラス	`class T(TestCase): def test_x(self): self.assertEqual(a, b)`		05 9. 自動のテスト
テストで要求を送る	`self.client.post("/login/", {...})`		05 9. 自動のテスト
テストでログインする	`self.client.force_login(user)`		＋ 合わせて覚える
エラーになるはずを確かめる	`with self.assertRaises(ValidationError):`		＋ 合わせて覚える